Literature Review: AI Agent Security Gateway

A Runtime Security Layer for Detecting Prompt Injection, Tool Poisoning, Unauthorized Tool Use, and Runtime Threats Before AI Agents Execute Tool Calls

Compiled July 2026. All papers below were verified via web search against publisher pages (ACM DL, USENIX, NDSS, IEEE Xplore, ICLR/NeurIPS/ICML OpenReview) or arXiv. Citation counts are approximate (Semantic Scholar / Google Scholar snapshots at time of writing) and will keep growing — treat them as ballpark, not precise.

PART 1 — ANNOTATED PAPER LIST

Papers are grouped by function within your architecture: **(A) Threat foundations**, **(B) Benchmarks/evaluation environments**, **(C) Runtime defense architectures**, **(D) Detection/classification methods**, **(E) Alignment/training-time defenses**, **(F) Adjacent attack surfaces (RAG, jailbreaks) you must defend against**.

A. THREAT FOUNDATIONS

1. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection

- **Authors:** Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz
- **Year:** 2023
- **Venue:** AISec '23 — 16th ACM Workshop on Artificial Intelligence and Security, co-located with **ACM CCS 2023**
- **Publisher:** ACM
- **DOI:** 10.1145/3605764.3623985 · arXiv:2302.12173
- **GitHub:** github.com/greshake/llm-security
- **Citations:** ~900–1,000+ (one of the most cited papers in the field)
- **Research Problem:** LLM-integrated applications blur the line between "instructions" and "data." No one had systematically shown that an attacker who cannot talk to the LLM directly can still compromise it by planting instructions in content the LLM later retrieves (emails, web pages, search results).
- **Proposed Solution:** Introduces **indirect prompt injection (IPI)** as a new attack class; builds a taxonomy (data theft, worming/self-propagation, ecosystem contamination, availability attacks) from a computer-security lens; demonstrates working exploits against Bing Chat/GPT-4 and synthetic LangChain apps.
- **Key Contributions:** First formal taxonomy of IPI; first real-world proof-of-concept against a production system (Bing Chat); coined the "prompt injection = SQL injection analogy" that now underlies most defenses (StruQ, CaMeL, instruction hierarchy).

- **Strengths:** Extremely influential; grounded in classical security taxonomy (STRIDE-adjacent thinking); real-world validated, not just synthetic.
- **Weaknesses:** No proposed defense/architecture — purely an attack/taxonomy paper; predates tool-calling APIs and MCP, so its examples are plugin/browsing-era.
- **Research Gap:** Does not address *runtime enforcement* — leaves "how do we stop this at the point of tool execution" fully open. This is precisely your thesis's gap.
- **Relation to your project:** This is the **threat model chapter** of your gateway. Every attack category your gateway must detect (data exfiltration via tool call, worming across agents, unauthorized action) traces back to this taxonomy.
- **Verdict: MUST READ**

2. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions

- **Authors:** Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, Alex Beutel
- **Year:** 2024
- **Venue:** OpenAI technical report / arXiv (widely cited, treated as a landmark industry paper; also under OpenReview)
- **Publisher:** OpenAI / arXiv:2404.13208
- **GitHub:** No official release (dataset generation method only)
- **Citations:** ~500+
- **Research Problem:** LLMs treat system prompts, user prompts, and third-party tool/web content as equally "privileged" tokens — there is no notion of trust level, unlike an OS's kernel/user-mode separation.
- **Proposed Solution:** A training-time **instruction hierarchy** (System > User > Tool/third-party data) enforced via synthetic data generation + fine-tuning (SFT/RLHF on GPT-3.5), teaching the model to treat lower-tier content as *data*, not *instructions*, when it conflicts with higher-tier goals.
- **Key Contributions:** First systematic OS-privilege analogy applied to LLM training; large robustness gains on unseen attack types with minimal utility loss; became the conceptual seed for almost every later "trust tier" architecture (CaMeL's privileged/quarantined LLM split, StruQ's channel separation, Progent's privilege DSL).
- **Strengths:** Backed by a frontier lab with production-scale evaluation; broadly generalizes to attack types not seen in training.
- **Weaknesses:** A *model-level* (probabilistic) fix, not a *deterministic* one — the authors themselves note it is not adversarially robust; doesn't solve tool-call authorization, only text-following behavior.
- **Research Gap:** No enforcement mechanism outside the model weights — a determined adaptive attacker can still find inputs that violate the hierarchy (as later

shown by adaptive-attack papers). This motivates deterministic, system-level gateways like your project.

- **Relation to your project:** Defines the **trust-tier abstraction** your gateway should encode explicitly (rather than hoping the model internalizes it) — i.e., your gateway is the "enforcement layer" this paper is missing.
- **Verdict: MUST READ**

3. Universal and Transferable Adversarial Attacks on Aligned Language Models (GCG)

- **Authors:** Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, Matt Fredrikson
- **Year:** 2023
- **Venue:** arXiv preprint (widely treated as foundational; underlies later ICML/NeurIPS adversarial-robustness work); code release paired with CMU/CAIS
- **Publisher:** arXiv:2307.15043
- **GitHub:** github.com/llm-attacks/llm-attacks (also `nanogcg`)
- **Citations:** ~2,500+ (one of the most cited LLM-security papers overall)
- **Research Problem:** Jailbreaks previously required human ingenuity and were brittle. Is there an automatic, transferable, optimization-based way to break alignment?
- **Proposed Solution: Greedy Coordinate Gradient (GCG)** — a gradient-guided discrete search that finds adversarial suffixes maximizing the likelihood of an affirmative (harmful) response; suffixes transfer across models (Vicuna → GPT, Claude, Bard, Llama-2).
- **Key Contributions:** Establishes that *any* content channel (including tool outputs / retrieved documents in your gateway's threat model) can carry optimization-crafted adversarial strings, not just human-readable injections; became the default "adaptive attacker" baseline that every defense paper (StruQ, SecAlign, DataSentinel, CaMeL) must evaluate against.
- **Strengths:** Rigorous, reproducible, open-sourced; demonstrated real transferability to production black-box systems.
- **Weaknesses:** Focused on jailbreaking harmful-content generation, not tool-calling/agentic actions specifically — you must extend the threat model yourself to "adversarial suffixes embedded in tool descriptions/outputs."
- **Research Gap:** No agentic/tool-execution evaluation — a clear extension opportunity (optimization-based tool-poisoning attacks, which MCPTox and others are just beginning to explore).
- **Relation to your project:** This is the **adaptive-attacker baseline** your gateway's evaluation must survive. Any detector you build (heuristic or ML-based) should be stress-tested with GCG-style optimized injections, not just hand-written ones.
- **Verdict: MUST READ**

B. BENCHMARKS / EVALUATION ENVIRONMENTS

4. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents

- **Authors:** Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, Florian Tramèr
- **Year:** 2024
- **Venue: NeurIPS 2024**, Datasets and Benchmarks Track (developed with Google DeepMind / Invariant Labs collaboration)
- **Publisher:** NeurIPS Proceedings
- **DOI/Link:** proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54
- **GitHub:** github.com/ethz-spylab/agentdojo
- **Citations:** ~250+ (rapidly rising; became the de facto standard within a year)
- **Research Problem:** Existing prompt-injection tests were static, unrealistic, or measured attack success without measuring whether the agent could still do its job (utility). No standard joint utility/security benchmark existed for tool-calling agents.
- **Proposed Solution:** A dynamic benchmark with 97 realistic tasks and 629 security test cases across 4 environments (Workspace/email, Slack, Travel, Banking), pairing each "user task" with an "injection task," and scoring via **formal utility checks on environment state** (not LLM-judged), so utility and security are measured jointly.
- **Key Contributions:** The standard evaluation harness for agentic prompt-injection research today; introduced the utility-under-attack metric now used by CaMeL, Progent, MELON, PromptArmor, and dozens of others.
- **Strengths:** Realistic multi-tool environments; extensible; actively maintained; used as the primary evaluation target by nearly every 2025 defense paper you'll cite.
- **Weaknesses:** Only 4 application domains; attacks are still largely hand-crafted (not fully adaptive/optimization-based) in the base release; doesn't yet model MCP-style dynamic tool discovery/poisoning.
- **Research Gap:** No first-class support for **tool-poisoning** (malicious tool *descriptions*, as opposed to malicious tool *outputs*) — a gap your gateway's evaluation should fill, since MCP tool poisoning is a distinct threat class AgentDojo wasn't designed for.
- **Relation to your project:** This should be your **primary evaluation benchmark** — build your gateway as a middleware layer and report Attack Success Rate (ASR) and Utility-under-Attack on AgentDojo exactly as CaMeL/Progent/MELON do, to make your results directly comparable to SOTA.
- **Verdict: MUST READ**

5. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents

- **Authors:** Qiusi Zhan, Zhixiang Liang, Zifan Ying, Daniel Kang
- **Year:** 2024
- **Venue: Findings of ACL 2024**
- **Publisher:** Association for Computational Linguistics
- **DOI:** 10.18653/v1/2024.findings-acl.624
- **GitHub:** github.com/uiuc-kang-lab/InjecAgent
- **Citations:** ~300+
- **Research Problem:** Need a large, systematic benchmark specifically measuring how often tool-integrated agents follow malicious instructions embedded in tool outputs (17 user tools × 62 attacker tools = 1,054 cases).
- **Proposed Solution:** Synthetic-but-systematic benchmark generation pipeline (GPT-4-assisted) covering "direct harm" and "data exfiltration" attack intents; evaluates 30 agent configurations, including a "hacking prompt" enhanced setting.
- **Key Contributions:** First large-scale, tool-diverse IPI benchmark; showed ReAct-prompted GPT-4 has a 24% baseline ASR, nearly doubling under enhanced attacks — widely cited figure establishing the severity of the problem.
- **Strengths:** Broad tool coverage (62 distinct "attacker tools"); simple to run; became a standard secondary benchmark alongside AgentDojo.
- **Weaknesses:** Shorter/simpler contexts than AgentDojo (avg. 1,033 tokens vs 3,823), making some defenses look artificially strong; single-turn oriented, less realistic multi-step agent trajectories.
- **Research Gap:** Lacks joint utility measurement (pure security-only benchmark) — doesn't penalize a defense that simply refuses everything.
- **Relation to your project:** Use as a **secondary/cross-validation benchmark** — a gateway that only performs well on AgentDojo but fails on InjecAgent's shorter, more numerous test cases would suggest overfitting to one context length.
- **Verdict: MUST READ**

6. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents

- **Authors:** Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, Yongfeng Zhang
- **Year:** 2025
- **Venue: ICLR 2025**
- **Publisher:** OpenReview / ICLR Proceedings
- **Link:** openreview.net/forum?id=V4y0CpX4hK · arXiv:2410.02644

- **GitHub:** github.com/agiresearch/ASB
- **Citations:** ~100+ (fast-growing)
- **Research Problem:** No benchmark unified prompt injection, **memory poisoning**, and a new **Plan-of-Thought (PoT) backdoor attack** across the full agent pipeline (system prompt, user prompt, tool use, memory retrieval) in one framework.
- **Proposed Solution:** 10 scenarios (finance, e-commerce, autonomous driving, etc.), 10 agents, 400+ tools, 27 attack/defense method types, 7 metrics, built atop the AIOS agent operating system.
- **Key Contributions:** Broadest attack-surface coverage to date (DPI + IPI + memory poisoning + backdoors + mixed attacks); highest reported ASR of 84.3% shows current defenses are largely inadequate; the PoT backdoor attack is a novel contribution (hidden triggers in system prompts).
- **Strengths:** Most comprehensive taxonomy-to-benchmark mapping available; directly usable as a unified evaluation harness for a gateway covering multiple threat classes (your project's stated scope).
- **Weaknesses:** Breadth over depth — individual scenarios are less deeply realistic than AgentDojo's; evaluation across 13 LLM backbones is compute-heavy to reproduce fully.
- **Research Gap:** Memory-poisoning and backdoor coverage is still shallow relative to prompt injection; doesn't address MCP-specific tool poisoning (tool *metadata* attacks) as a first-class category.
- **Relation to your project:** Because your gateway targets **prompt injection + tool poisoning + unauthorized tool use + runtime threats** as four distinct classes, ASB is the single benchmark whose taxonomy most closely mirrors your own scope — use it to structure your own threat-class breakdown and as a second major evaluation target after AgentDojo.
- **Verdict: MUST READ**

7. Identifying the Risks of LM Agents with an LM-Emulated Sandbox (ToolEmu)

- **Authors:** Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, Tatsunori Hashimoto
- **Year:** 2024
- **Venue: ICLR 2024** (Spotlight)
- **Publisher:** OpenReview / ICLR Proceedings
- **Link:** openreview.net/forum?id=GEcwtMk1uA · arXiv:2309.15817
- **GitHub:** github.com/ryoungj/ToolEmu
- **Citations:** ~250+
- **Research Problem:** Manually implementing tools and sandboxes to test agent safety doesn't scale; long-tail, high-stakes risks (financial loss, data leakage) are expensive to

discover via manual red-teaming.

- **Proposed Solution:** Use an LM itself to **emulate tool execution** from tool specs alone (no real implementation needed), paired with an LM-based automatic safety evaluator; 36 high-stakes toolkits, 144 test cases.
- **Key Contributions:** Demonstrated even the safest agents fail 23.9% of the time on high-stakes tasks; showed automated emulation can find failures 30x faster than manual sandbox construction (15 min vs 8 hrs per case); established "instruction underspecification" (benign-but-ambiguous user asks) as a distinct risk class from adversarial injection.
- **Strengths:** Scalable methodology; useful not just for attacks but for **benign risk/misuse** discovery, which is relevant to your "unauthorized tool use" category even without an adversary.
- **Weaknesses:** LM-emulated environments are not a perfect substitute for real tool execution (68.8% validity rate found in human evaluation, meaning ~31% false positives); not adversarial-attacker focused, more "agent safety under ambiguity."
- **Research Gap:** Doesn't address adversarial prompt injection at all — purely a benign-misuse/ambiguity risk framework, complementary to (not overlapping with) InjecAgent/AgentDojo.
- **Relation to your project:** Useful for the **"unauthorized tool use" / runtime-threat** pillar of your gateway — many unauthorized actions arise not from an attacker but from an underspecified user request combined with an over-eager agent; ToolEmu gives you a scalable way to generate such test cases pre-deployment.
- **Verdict: IMPORTANT**

8. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models

- **Authors:** Wei Zou, Runpeng Geng, Binghui Wang, Jinyuan Jia
- **Year:** 2025 (USENIX Security 25 proceedings; arXiv preprint 2024)
- **Venue: 34th USENIX Security Symposium (USENIX Security '25)**
- **Publisher:** USENIX Association
- **Link:** usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag · arXiv:2402.07867
- **GitHub:** github.com/sleeepeer/PoisonedRAG
- **Citations:** ~350+ (rapidly rising; "Highly Influential Citations" flagged by Semantic Scholar)
- **Research Problem:** RAG's knowledge database is an unexamined new attack surface — can an attacker inject a *small number* of malicious documents to force an attacker-chosen answer?

- **Proposed Solution:** Formalizes knowledge corruption as an optimization problem; proposes black-box and white-box solutions; achieves ~90% ASR injecting just 5 malicious texts into a multi-million-document corpus.
- **Key Contributions:** First systematic RAG poisoning attack with formal optimization framing; shows existing defenses (perplexity filtering, duplicate detection) are largely ineffective.
- **Strengths:** Rigorous USENIX-quality methodology; both threat models (black/white box) covered; strong, reproducible results.
- **Weaknesses:** Targets QA-style RAG, not tool-calling agents directly — you must bridge the gap to "poisoned memory/RAG feeding into a tool-call decision."
- **Research Gap:** No defense proposed by the authors — an open problem your gateway could partially address by treating retrieved/RAG content as untrusted input requiring the same provenance-tagging your gateway applies to tool outputs.
- **Relation to your project:** Directly relevant to the **"runtime threats"** pillar — if your agent's tool-call decisions are influenced by RAG-retrieved context, this paper is your threat model for that data path, and its optimization-based attacker is another adaptive baseline to test against.
- **Verdict: IMPORTANT**

C. RUNTIME DEFENSE ARCHITECTURES (closest prior art to your gateway)

9. Defeating Prompt Injections by Design (CaMeL)

- **Authors:** Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, Florian Tramèr
- **Year:** 2025
- **Venue: arXiv preprint / Google DeepMind technical report** (highly influential industry paper; not yet at a peer-reviewed venue as of writing, but treated as a landmark by the research community — Simon Willison called it "the first credible prompt injection mitigation" leaning on real security engineering)
- **Publisher:** Google DeepMind / arXiv:2503.18813
- **GitHub:** github.com/google-research/camel-prompt-injection
- **Citations:** ~150+ (fast-growing given recency)
- **Research Problem:** Nearly all prior defenses are probabilistic (a fine-tuned model that is "usually" robust). Attackers only need the 1% failure case — application security requires deterministic guarantees, not 99% detection.
- **Proposed Solution: CaMeL** ("CApabilities for MachinE Learning") — dual-LLM architecture: a **Privileged LLM** plans from the trusted user query and emits a program; a **Quarantined LLM** processes untrusted data with *no tool access*; a custom Python interpreter tracks data provenance/capabilities on every value and enforces

security policies *before* each tool call, blocking or requiring confirmation for unauthorized data flows.

- **Key Contributions:** Provably solves 77% of AgentDojo tasks *with formal security guarantees* (vs. 84% undefended, no guarantees) — the first credible security-engineering (not ML) solution; explicitly separates control-flow from data-flow, mirroring classical taint tracking/capability security.
- **Strengths:** Deterministic, model-agnostic (works without retraining/fine-tuning the underlying LLM); directly demonstrates the "policy enforcement before tool execution" paradigm your project is built around.
- **Weaknesses:** Requires codifying policies (developer/user burden); risk of confirmation-prompt fatigue; utility drop (84%→77% on AgentDojo) shows a real security/utility tradeoff; no production-grade open release yet at initial publication.
- **Research Gap:** Assumes tool descriptions/schemas themselves are trustworthy — does **not** address tool-poisoning (malicious tool *metadata*), which is squarely in your project's scope; also doesn't address multi-agent/cross-agent propagation.
- **Relation to your project: This is the single closest prior-art architecture to what you are building.** Your gateway is essentially generalizing CaMeL's "capability check before tool execution" idea into a protocol-level (MCP-aware), pluggable middleware that also covers tool poisoning and unauthorized-use detection that CaMeL doesn't. Position your contribution explicitly against CaMeL in your related work.
- **Verdict: MUST READ**

10. Progent: Programmable Privilege Control for LLM Agents

- **Authors:** Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, Dawn Song
- **Year:** 2025
- **Venue:** arXiv preprint (UC Berkeley / Dawn Song's group; widely cited within months, github stars indicate strong community traction)
- **Publisher:** arXiv:2504.11703
- **GitHub:** Yes (linked from HuggingFace paper page)
- **Citations:** ~60–100 (fast-growing; only months old)
- **Research Problem:** Least-privilege enforcement for agents is conceptually appealing but nobody had built a deployable, expressive, and *automatically-generated* privilege system.
- **Proposed Solution:** A **domain-specific language (DSL)** for expressing fine-grained, per-tool-call privilege policies; policies are auto-generated by an LLM from the user's task and then updated dynamically via an **SMT solver** that only allows monotonic narrowing without explicit approval (preventing silent privilege escalation); deterministic enforcement at every tool call.

- **Key Contributions:** First privilege-control mechanism purpose-built for agents; evaluated on **three benchmarks at once** (AgentDojo, ASB, AgentPoison) showing strong security with high utility; validated in real frameworks (LangChain, OpenAI Agents SDK) — i.e., not just a research toy.
- **Strengths:** Deterministic, SMT-verified security guarantee on policy updates; practical, modular, low integration cost; directly deployable in the exact frameworks your gateway would sit alongside.
- **Weaknesses:** Requires an LLM to bootstrap policies, reintroducing a probabilistic component at initialization (though the *runtime check* itself is deterministic); DSL authoring burden for complex policies; adaptive-attack resilience against the *policy-generation* LLM is a stated but not fully solved concern.
- **Research Gap:** Focuses on privilege/authorization, not on *detecting* prompt injection or tool-poisoning content per se — a natural complement to (not a replacement for) a detection-layer component in your gateway.
- **Relation to your project:** This is your **closest architectural template for the "unauthorized tool use" pillar**. Your gateway's policy-enforcement component should be benchmarked directly against Progent.
- **Verdict: MUST READ**

11. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems

- **Authors:** Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, Umar Iqbal
- **Year:** 2025
- **Venue: NDSS Symposium 2025**
- **Publisher:** Internet Society (NDSS)
- **Link:** ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems
- **GitHub:** github.com/llm-platform-security/SecGPT (project name SecGPT, paper name IsolateGPT)
- **Citations:** ~50–100 (Tier-1 venue, growing steadily)
- **Research Problem:** LLM-app ecosystems (ChatGPT plugins, agent tool marketplaces) resemble early-2000s operating systems with **no isolation** between third-party apps and the system — a third-party app/tool can freely read/write shared state.
- **Proposed Solution: IsolateGPT / SecGPT** — an OS-inspired execution-isolation architecture: each app/tool executes in its own isolated "compartment," and inter-app or app-system communication is only allowed through well-defined interfaces requiring explicit user permission.
- **Key Contributions:** First systems-security (isolation-based, not detection-based) architecture purpose-built for LLM agent ecosystems; defends against app compromise, data stealing, inadvertent data exposure, and uncontrolled system

alteration simultaneously; <30% performance overhead for 75.7% of queries — shows isolation is practically deployable.

- **Strengths:** Tier-1 (NDSS) rigor; directly reusable systems-security pattern (process isolation → agent/tool isolation); open-sourced and integrated into LangChain and LlamaIndex.
- **Weaknesses:** A 2026 follow-up study found concrete attacks against IsolateGPT itself (Execution Flow Disruption, Execution Manager Hijack, Planner Manipulation) exploiting the interleaved plan-execute design and assuming too-weak an adversary — i.e., **isolation alone is not sufficient**, an important finding for your design.
- **Research Gap:** The 2026 attack paper against it explicitly shows the planning phase's integrity is *not* protected by isolation — this is a validated open problem your gateway can target (e.g., cryptographically or structurally binding tool descriptions to the plan so a poisoned tool can't rewrite the planner's intent).
- **Relation to your project:** Provides the **systems-security half** of your architecture (execution isolation) to pair with CaMeL's capability/data-flow half and Progent's privilege half. Also gives you a **known, published attack** against a published defense to cite as motivation for why detection *and* isolation *and* privilege control together (your gateway's multi-layer design) are necessary.
- **Verdict: MUST READ**

12. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning

- **Authors:** Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, Bo Li
- **Year:** 2025
- **Venue: ICML 2025**
- **Publisher:** PMLR / ICML Proceedings
- **Link:** icml.cc/virtual/2025/poster/46569 · arXiv:2406.09187
- **GitHub:** github.com/guardagent/code
- **Citations:** ~80–150
- **Research Problem:** Traditional LLM guardrails only handle *textual harm* (toxicity, jailbreaks) — they cannot check whether an *agent's actions* comply with structured safety/privacy policies (e.g., "a nurse role may only read, not write, patient records").
- **Proposed Solution: GuardAgent** — literally an agent-as-a-gateway: it (1) analyzes natural-language guard requests to produce a task plan, then (2) compiles that plan into executable **guardrail code**, which is run deterministically against the target agent's inputs/outputs at each step; supplemented by a memory module of past guard-request/code pairs for few-shot reuse.

- **Key Contributions:** First "guard agent" pattern — treating safety enforcement itself as a code-generation + code-execution problem rather than a classifier; introduces two new benchmarks (EICU-AC for healthcare access control, Mind2Web-SC for web-agent safety); 98%/83% guardrail accuracy on the two benchmarks respectively.
- **Strengths:** Directly literal precedent for a "security gateway" pattern — sits *between* the user/agent and the target agent's actions; code-based enforcement gives deterministic, auditable decisions (not just an LLM's yes/no).
- **Weaknesses:** Relies on an LLM to translate policies into code, so correctness of the *generated guardrail code* is itself a trust/verification question; evaluated only on two narrow domains (healthcare, web agents) — generality to arbitrary tool ecosystems (e.g., MCP) untested; guard-agent latency/cost overhead of a second LLM call per action isn't deeply analyzed.
- **Research Gap:** No adversarial evaluation against attackers specifically trying to fool the *guard agent itself* (e.g., prompt-inject the guard's code-generation step) — an important vulnerability class for you to test in your own gateway.
- **Relation to your project:** This is the paper whose **name and pattern most literally match "security gateway."** Cite it as the closest precedent for the architectural pattern (a separate reasoning agent that checks compliance before/after each action), and explicitly differentiate your work by making enforcement policy-driven/deterministic rather than LLM-code-generated at runtime, and by targeting *tool-call interception* specifically rather than general guard requests.
- **Verdict: MUST READ**

13. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents

- **Authors:** Haoyu Wang, Christopher M. Poskitt, Jun Sun
- **Year:** 2026 (ICSE 2026 Research Track; arXiv preprint 2025)
- **Venue: ICSE 2026** (International Conference on Software Engineering — Tier-1 software engineering venue)
- **Publisher:** ACM/IEEE ICSE Proceedings
- **Link:** conf.researchr.org/details/icse-2026/icse-2026-research-track/29 · arXiv:2503.18666
- **GitHub:** Referenced in paper (SMU research group release)
- **Citations:** ~30–60 (very recent)
- **Research Problem:** Most agent-safety mechanisms are pre-execution risk assessments (static analysis before running) with **no active runtime enforcement** — meaning an agent can still deviate mid-execution with nothing stopping it.
- **Proposed Solution: AgentSpec**, a lightweight DSL where users define rules with **triggers, predicates, and enforcement mechanisms** that are checked and enforced

live during agent execution, across code agents, embodied agents, and autonomous-vehicle agents.

- **Key Contributions:** Prevents >90% of unsafe code-agent executions, 100% of hazardous embodied-agent actions, and 100% law-violation prevention for AVs, all at millisecond-level overhead; demonstrates automatic rule generation from LLMs (o1) with measured precision/recall (95.56%/70.96%).
- **Strengths:** Cross-domain generality (not just chat agents — code, robotics, driving); genuinely lightweight/low-latency, a key requirement for a *gateway* that must sit in the hot path of every tool call; software-engineering rigor (ICSE is a top venue for runtime verification).
- **Weaknesses:** No trajectory-level/foresight reasoning (can't anticipate multi-step consequences, only single-action triggers per the authors' own limitations section); not specifically designed against adversarial prompt injection — closer to a general safety-rule engine.
- **Research Gap:** Explicitly flagged by the authors: no probabilistic/trajectory-based reachability analysis for detecting *slow-building* multi-step attacks — directly relevant since sophisticated tool-poisoning/IPI attacks often unfold over several turns.
- **Relation to your project:** Gives you the **runtime-enforcement engine pattern** (trigger/predicate/action DSL) at low latency, complementary to Progent's privilege DSL and CaMeL's capability tracking — a strong architectural building block for the enforcement half of your gateway (as opposed to the detection half).
- **Verdict: IMPORTANT**

D. DETECTION / CLASSIFICATION METHODS (the "detect before execution" component)

14. DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks

- **Authors:** Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, Neil Zhenqiang Gong
- **Year:** 2025
- **Venue: 46th IEEE Symposium on Security and Privacy (IEEE S&P / "Oakland" 2025)**
- **Publisher:** IEEE
- **DOI:** 10.1109/SP61157.2025.00250
- **Citations:** ~40–80 (very recent, Tier-1 venue)
- **Research Problem:** Existing prompt-injection *detectors* (classifiers, canary tokens, perplexity filters) are trained against known attacks and collapse against adaptive attackers who specifically optimize to evade the detector.
- **Proposed Solution:** Formulates detection as a **minimax game**: fine-tune a detector LLM to be robust against the *worst-case* adaptive injected prompt, solved via

alternating gradient-based optimization between an inner maximization (find the hardest-to-detect injection) and outer minimization (improve the detector).

- **Key Contributions:** State-of-the-art detection accuracy against both known and adaptive/optimization-based injections — directly outperforms prior heuristic detectors; the game-theoretic training procedure is a reusable general recipe.
- **Strengths:** Rigorous adversarial evaluation (this is exactly what most competing detectors lack); Tier-1 peer review; from a well-established security research group (Neil Gong's lab, several papers in this list).
- **Weaknesses:** Requires a dedicated fine-tuned detector model (extra inference cost per tool call, a latency concern for a gateway sitting in the hot path); binary detection only — doesn't localize *which* part of the input is malicious (a related paper, AttnTrace/TracLLM, addresses attribution).
- **Research Gap:** No integration with an enforcement/action layer — it tells you "this input is contaminated," not "block/allow/modify this specific tool call," which is exactly the gap your gateway should fill (detection → enforcement pipeline).
- **Relation to your project:** This is your **state-of-the-art detection baseline** for the "detect prompt injection" pillar of your gateway; you should compare your detector's ASR-reduction and false-positive rate directly against DataSentinel's published numbers.
- **Verdict: MUST READ**

15. Formalizing and Benchmarking Prompt Injection Attacks and Defenses

- **Authors:** Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, Neil Zhenqiang Gong
- **Year:** 2024
- **Venue: 33rd USENIX Security Symposium (USENIX Security '24)**
- **Publisher:** USENIX Association
- **Link:** usenix.org/conference/usenixsecurity24/presentation/liu-yupei
- **GitHub:** github.com/liu00222/Open-Prompt-Injection
- **Citations:** ~200+
- **Research Problem:** Before this paper, prompt injection attacks/defenses were described ad hoc with no shared formal vocabulary, making comparison across papers unreliable.
- **Proposed Solution:** A general **mathematical framework** in which every known attack (naive concatenation, escape characters, context ignoring, fake completion, combined attacks) is a special case; systematically benchmarks 5 attacks × 10 defenses × 10 LLMs × 7 tasks.
- **Key Contributions:** The formalism used (implicitly or explicitly) by nearly every later paper in this list; a new combined attack constructed from the formal framework

outperforms all prior individual attacks; public benchmarking platform (Open-Prompt-Injection).

- **Strengths:** Clean, reusable formal definitions; huge combinatorial evaluation surface, giving strong statistical confidence; open-source benchmark platform lowers the barrier for you to reproduce baselines.
- **Weaknesses:** Task-level (single LLM call) framing, not full multi-step agentic/tool-calling framing — AgentDojo/InjecAgent extend this to agents specifically.
- **Research Gap:** Does not model tool-calling agents or multi-turn state — a direct segue to InjecAgent/AgentDojo, both of which cite this as their formal foundation.
- **Relation to your project:** Use this paper's **formal attack taxonomy** (not its agent coverage) as the mathematical backbone of your threat model chapter, then extend it explicitly to the tool-call setting as your novel contribution.
- **Verdict: MUST READ**

16. StruQ: Defending Against Prompt Injection with Structured Queries

- **Authors:** Sizhe Chen, Julien Piet, Chawin Sitawarin, David Wagner
- **Year:** 2025 (accepted; arXiv preprint 2024)
- **Venue: 34th USENIX Security Symposium (USENIX Security '25)**
- **Publisher:** USENIX Association
- **Link:** usenix.org/conference/usenixsecurity25/presentation/chen-sizhe
- **GitHub:** github.com/Sizhe-Chen/StruQ
- **Citations:** ~150+
- **Research Problem:** Prompt and data are concatenated into a single token stream with no structural separation, so the model has no reliable signal for "this part is trusted instruction, this part is untrusted data."
- **Proposed Solution: Structured queries** — a secure front-end formats prompt/data into two distinct channels using special reserved tokens/delimiters, paired with a model **fine-tuned via "structured instruction tuning"** to only follow instructions in the prompt channel, explicitly trained (via synthetic examples) to ignore instructions appearing in the data channel.
- **Key Contributions:** A concrete, reproducible instantiation of "instruction hierarchy" via input formatting + targeted fine-tuning; strong empirical robustness with minimal utility loss; directly inspired the later **SecAlign** (preference-optimization version) and **Meta SecAlign** (production-scale version).
- **Strengths:** USENIX-quality rigor; open-source, reproducible; simple front-end architecture is easy to integrate into any gateway (channel-tagging is exactly the kind of pre-processing a gateway would do before an LLM call).
- **Weaknesses:** Requires retraining/fine-tuning the underlying LLM (not usable as a drop-in gateway for closed models like GPT-4/Claude without provider cooperation);

GCG-style optimization attacks remain partially effective.

- **Research Gap:** Channel separation alone doesn't stop tool-call-level authorization issues (an "authorized" instruction can still request an unauthorized action) — complementary to, not a substitute for, privilege-control papers like Progent.
- **Relation to your project:** Gives you the **input-formatting/pre-processing** pattern your gateway can apply to tool outputs and RAG content before they reach the agent LLM (tagging/structuring untrusted content), regardless of whether you fine-tune the underlying model.
- **Verdict: IMPORTANT**

17. SecAlign: Defending Against Prompt Injection with Preference Optimization

- **Authors:** Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, Chuan Guo
- **Year:** 2025
- **Venue: ACM CCS 2025** (ACM SIGSAC Conference on Computer and Communications Security)
- **Publisher:** ACM
- **Link:** ACM DL, pages 2833–2847
- **Citations:** ~50–100 (very recent Tier-1 venue)
- **Research Problem:** StruQ's fine-tuning approach still leaves measurable attack success; can **preference optimization** (DPO-style, contrasting desired vs. injected-instruction-following outputs) do better?
- **Proposed Solution:** SecAlign trains the model with preference pairs where "ignore injected instruction" is preferred over "follow injected instruction," reducing prompt-injection success to **under 10%** even against attacks stronger than those seen in training — the strongest reported robustness at publication.
- **Key Contributions:** First defense (per the authors) to push adaptive-attack success below 10% while preserving utility; the method (and its follow-up, Meta SecAlign) has been adopted at production scale (Meta).
- **Strengths:** Best-in-class robustness numbers at time of publication; CCS-level rigor; scalable methodology already validated in production (Meta SecAlign).
- **Weaknesses:** Same limitation family as StruQ — requires fine-tuning access to model weights, not applicable as a pure external gateway for closed-weight commercial APIs.
- **Research Gap:** Like StruQ, addresses instruction-following robustness, not tool-call authorization/isolation — your gateway's non-training-based, external-layer approach is explicitly complementary (and necessary for closed models where fine-tuning isn't possible).
- **Relation to your project:** Cite as the **current SOTA ceiling for model-level defenses**, and position your gateway as solving the *complementary, deployment-time* problem:

what do you do when you cannot fine-tune the model (e.g., calling GPT-4/Claude/Gemini via API)? This is a strong motivating contrast in your related work.

- **Verdict: IMPORTANT**

E. TOOL POISONING / MCP-SPECIFIC SECURITY (most directly on-topic for your gateway)

18. Model Context Protocol Threat Modeling and Analysis of Vulnerabilities to Prompt Injection with Tool Poisoning

- **Authors:** Charoes Huang et al. (4 authors, incl. Amin Milani Fard, corresponding)
- **Year:** 2026
- **Venue:** Journal article (MDPI — *note: journal-tier, not Tier-1 conference; included because it is one of the only systematic empirical studies of tool-poisoning defenses across real MCP clients*)
- **Publisher:** MDPI
- **DOI/Link:** mdpi.com/2624-800X/6/3/84 · arXiv:2603.22489
- **Citations:** New (2026) — not yet substantially cited
- **Research Problem:** MCP security research has focused almost entirely on **server-side** vulnerabilities; **client-side** validation of tool metadata is largely unstudied, despite tool poisoning being ranked #1 in the OWASP LLM Top 10 and scoring "Critical" (46.5/50) under DREAD analysis.
- **Proposed Solution:** STRIDE-based threat modeling covering 57 threats; empirical evaluation of **7 major MCP clients'** validation/defense behavior against tool poisoning; proposes a **multi-layered defense**: static metadata analysis, model decision-path tracking, behavioral anomaly detection, and user transparency mechanisms.
- **Key Contributions:** First systematic cross-client empirical comparison for MCP tool poisoning specifically; concrete, actionable multi-layer defense blueprint aligned almost exactly with a "gateway" design.
- **Strengths:** Directly on-topic (MCP + tool poisoning + client-side focus); practical, implementable recommendations; recent enough to reflect the current MCP ecosystem.
- **Weaknesses:** Journal (MDPI), not a Tier-1 security conference — lower peer-review bar than S&P/CCS/USENIX/NDSS; no working reference implementation released; empirical study is client-behavior focused, not a novel algorithmic detector.
- **Research Gap:** No proposed algorithm for the "model decision path tracking" or "behavioral anomaly detection" layers it recommends — purely a blueprint, not an implementation. **This is a direct implementation opportunity for your thesis.**
- **Relation to your project:** This paper's proposed **four-layer defense structure is almost identical to what your gateway should implement**, but nobody has built and

evaluated it — a strong "gap-filling" framing for your contribution.
- **Verdict: MUST READ** (for scope-fit, despite non-Tier-1 venue)

19. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers

- **Authors:** (arXiv 2025; full author list confirm via paper)
- **Year:** 2025
- **Venue:** arXiv preprint (benchmark paper; not yet at a peer-reviewed venue at time of writing, but highly relevant and increasingly cited in MCP-security follow-ups)
- **Publisher:** arXiv:2508.14925
- **Citations:** New, fast-growing given the surge in MCP-security papers citing it (e.g., "AIRGuard," "AgentDyn," others in this list)
- **Research Problem:** All prior IPI benchmarks (InjecAgent, AgentDojo) use **simulated** environments and injected payloads in tool *outputs* — but tool poisoning attacks malicious tool *descriptions/metadata*, a fundamentally different injection point, and prior benchmarks' payloads transfer poorly (near-0% ASR when repurposed).
- **Proposed Solution:** First benchmark testing tool poisoning against **45 real-world MCP servers** across 8 domains, with three distinct attack paradigms and design principles for crafting effective poisoned tool descriptions.
- **Key Contributions:** Empirically demonstrates tool poisoning is a genuinely distinct threat from IPI (not just a variant); shows current content-based safety alignment has a refusal rate below 3% against tool-poisoning payloads — a striking finding.
- **Strengths:** Real-world (not simulated) MCP servers — much higher ecological validity than any other benchmark in this list for the MCP-specific threat; directly usable as your gateway's primary tool-poisoning evaluation set.
- **Weaknesses:** arXiv preprint only at time of writing — not yet peer-reviewed at a top venue; narrower scope than AgentDojo/ASB (tool poisoning only, not the full IPI/memory/backdoor spectrum).
- **Research Gap:** No defense proposed — pure attack/benchmark paper, same gap structure as PoisonedRAG (attack surface established, defense wide open).
- **Relation to your project: This should be your primary tool-poisoning evaluation benchmark**, exactly analogous to how AgentDojo is your primary IPI evaluation benchmark. Report ASR reduction on MCPTox as a headline result.
- **Verdict: MUST READ**

F. SYSTEMATIZATION / SURVEY (included as the field's closest thing to a "definitive" synthesis)

20. Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents

- **Year:** 2025

- **Venue:** arXiv preprint (multi-institution, includes contributors referencing CaMeL/Foerster et al. 2026 — treat as a **position/challenges paper**, not a classic literature survey)
- **Publisher:** arXiv:2505.02077
- **Citations:** Growing; frequently cited as the framing paper for multi-agent security discussions
- **Research Problem:** Almost all prior work (including everything above) addresses **single-agent** security. What new threats and open problems emerge when *multiple* agents interact (agent-to-agent prompt propagation, cross-agent trust, emergent collusion)?
- **Proposed Solution:** Not a defense — a structured enumeration of open problems, including detailed case studies (e.g., a full walkthrough of CaMeL's design as an exemplar single-agent defense, then extending the discussion to multi-agent settings).
- **Key Contributions:** Clean case-study explanation of CaMeL (useful as a secondary/simplified reference for your defense chapter); identifies multi-agent-specific open problems (e.g., an injected instruction "worming" from one compromised agent to others it communicates with) that map directly onto Greshake et al.'s original "worming" attack category, now in an agentic multi-agent context.
- **Strengths:** Useful synthesis and forward-looking framing; good source of **novel research idea candidates** (see Part 5 below) since it explicitly enumerates unsolved problems.
- **Weaknesses:** Not empirical — no new experiments/benchmarks/defenses of its own; as a challenges/position paper rather than a rigorously peer-reviewed survey, treat its claims as directional rather than validated.
- **Research Gap:** Explicitly the paper's own point — multi-agent security is almost entirely open.
- **Relation to your project:** Useful for your **Future Work / Limitations** section — your gateway is naturally single-agent-per-instance; this paper helps you scope what's out of bounds for your thesis (multi-agent propagation) while acknowledging it as a natural extension.
- **Verdict: OPTIONAL** (useful for framing/future work, not for core architecture)

PART 2 — TOP 8 PAPERS RANKED FOR BUILDING AN AI AGENT SECURITY GATEWAY

Ranked by direct architectural relevance to **runtime interception before tool execution** — not by citation count or venue prestige alone.

Rank	Paper	Why it's ranked here
1	**Defeating Prompt Injections by Design (CaMeL)** — Google DeepMind, 2025	The closest existing system to "detect/enforce before tool execution" — capability tracking + data-flow policy checks at every tool call, with a formal security guarantee. Your architectural north star.
2	**Progent: Programmable Privilege Control for LLM Agents** — 2025	Directly operationalizes "unauthorized tool use" prevention via a deterministic, SMT-verified privilege DSL, evaluated across 3 benchmarks and 2 real agent frameworks.
3	**AgentDojo** — NeurIPS 2024	Your evaluation harness. Without this, you cannot make your results comparable to the rest of the field.
4	**GuardAgent** — ICML 2025	The literal "guard/gateway agent" pattern — plan-then-compile-to-code enforcement sitting between agent and action.
5	**IsolateGPT** — NDSS 2025	Systems-security/isolation half of your architecture, plus a documented, published *attack against it* that motivates why isolation alone is insufficient (motivating your multi-layer design).
6	**DataSentinel** — IEEE S&P 2025	SOTA detection component — the "detect" half of "detect … before AI Agents execute tool calls," adversarially validated.
7	**The Instruction Hierarchy** — OpenAI, 2024	The conceptual trust-tier abstraction (System > User > Tool/Third-party) your gateway's policy engine should encode explicitly at the protocol level.
8	**MCPTox** — 2025	The only benchmark with real-world MCP servers testing tool poisoning specifically — makes your gateway's tool-poisoning defense empirically falsifiable, not just theoretical.

(InjecAgent, ASB, PoisonedRAG, Not What You've Signed Up For, StruQ/SecAlign, and AgentSpec are all "MUST READ / IMPORTANT" per Part 1, but were ranked outside the top 8 here because they are either secondary benchmarks, foundational-but-non-architectural, or address a channel — RAG, model fine-tuning — one step removed from tool-call interception itself.)

PART 3 — READING ORDER

Phase 1 — Establish the threat model (read first, in this order):

1. Not What You've Signed Up For (Greshake et al.)
2. Formalizing and Benchmarking Prompt Injection Attacks and Defenses (Liu et al., USENIX '24)
3. Universal and Transferable Adversarial Attacks (GCG)
4. Model Context Protocol Threat Modeling / Tool Poisoning paper + MCPTox

Phase 2 — Understand how the field measures success (benchmarks): 5. InjecAgent (ACL '24) 6. AgentDojo (NeurIPS '24) 7. Agent Security Bench / ASB (ICLR '25) 8. ToolEmu (ICLR '24) — for non-adversarial risk framing 9. PoisonedRAG (USENIX '25) — if your gateway will touch RAG-fed agents

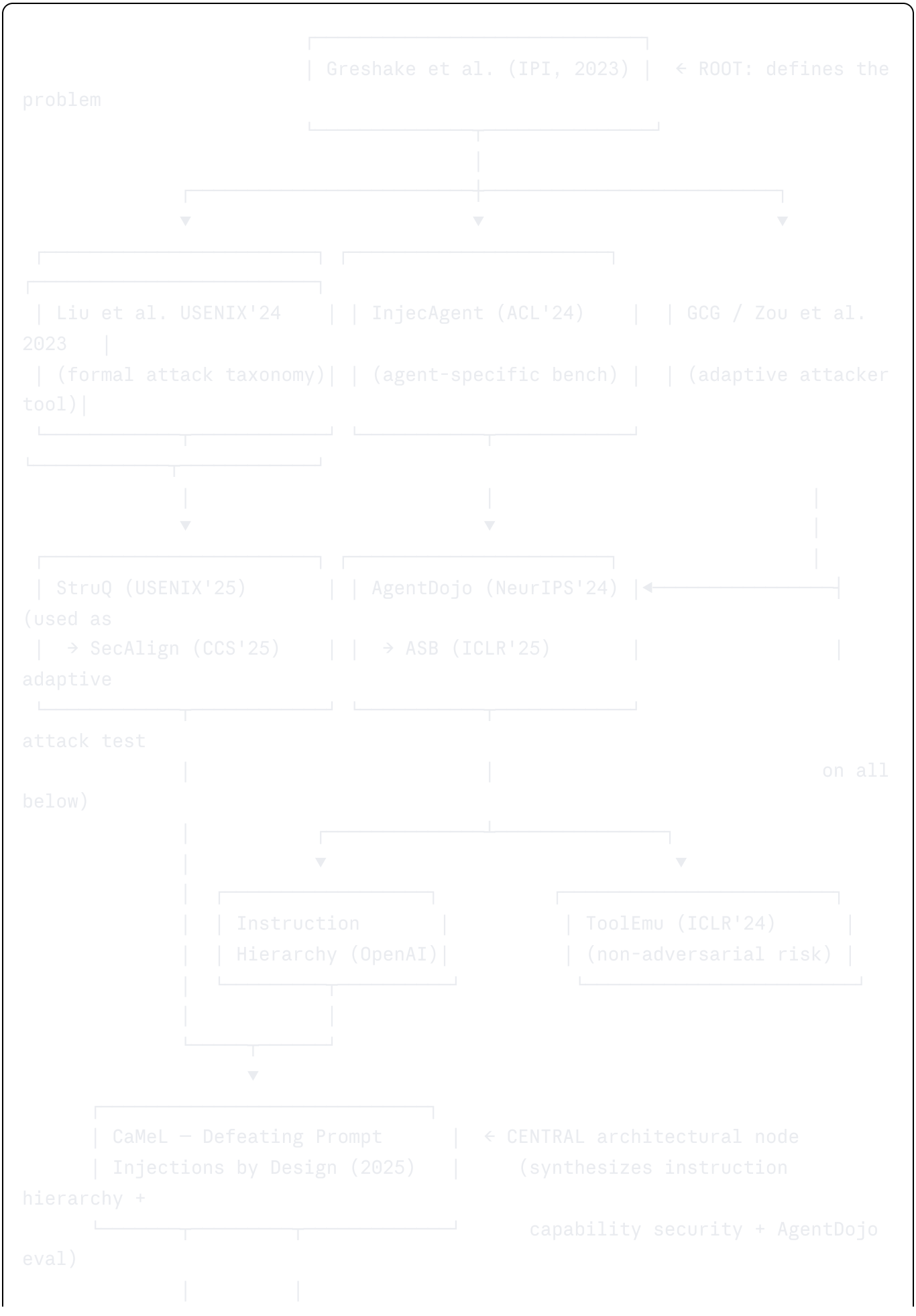
Phase 3 — Study the conceptual defense foundation: 10. The Instruction Hierarchy (OpenAI) 11. StruQ (USENIX '25) → SecAlign (CCS '25) — read as a pair, SecAlign is the successor

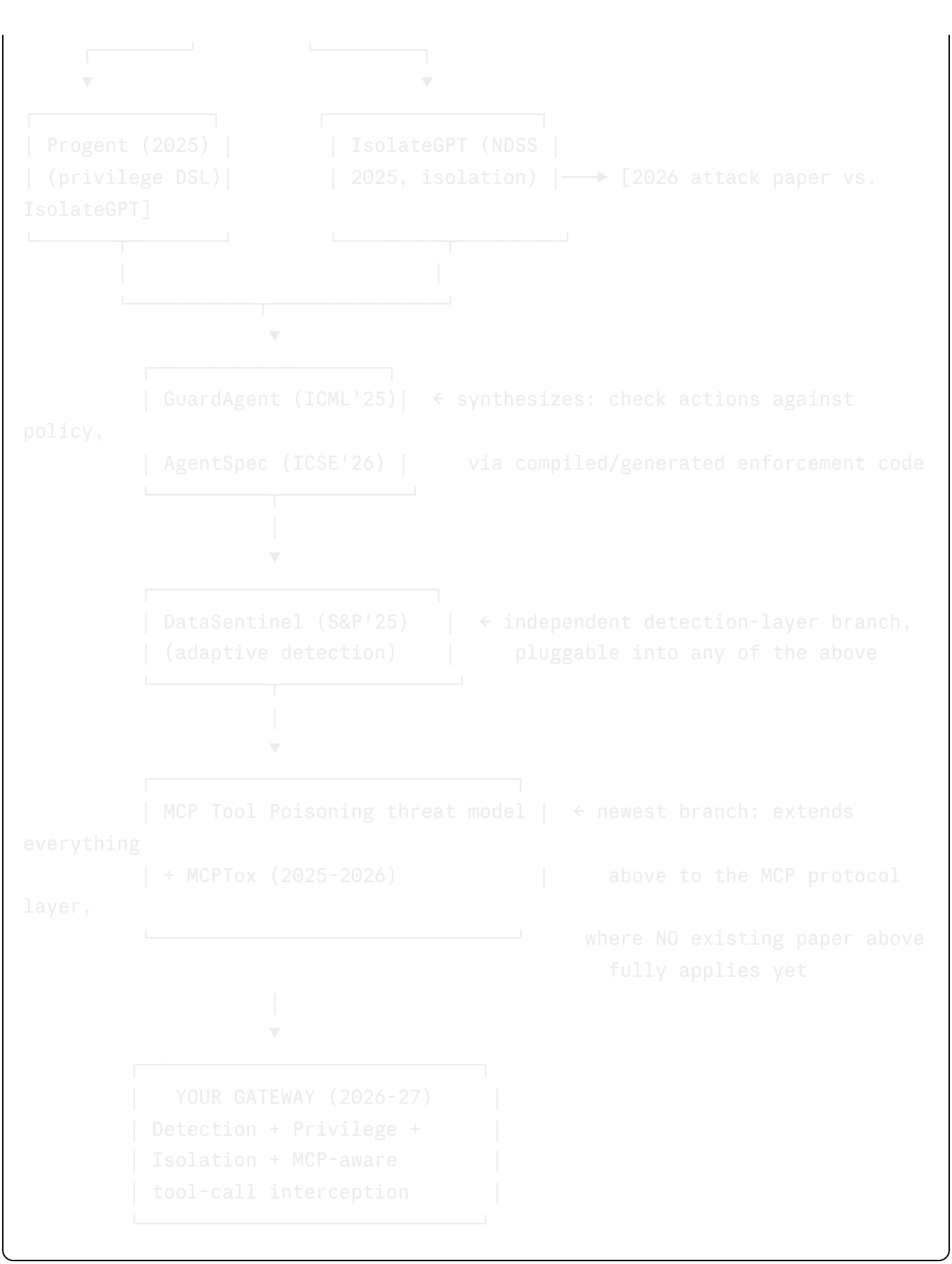
Phase 4 — Study the closest architectural prior art (core of your related work): 12. CaMeL / Defeating Prompt Injections by Design 13. Progent 14. IsolateGPT (+ note the 2026 attack paper against it) 15. GuardAgent 16. AgentSpec

Phase 5 — Study the detection layer you'll need to integrate: 17. DataSentinel

Phase 6 — Zoom out: 18. Open Challenges in Multi-Agent Security (for framing/future work)

PART 4 — KNOWLEDGE DEPENDENCY GRAPH





Reading implication: Greshake (2023) → Instruction Hierarchy (2024) → AgentDojo (2024) → CaMeL (2025) is the critical path. Everything else branches off or feeds into that spine.

PART 5 — MISSING RESEARCH OPPORTUNITIES

1. **No unified runtime gateway spans detection + privilege + isolation simultaneously.** Every existing system picks *one* layer: CaMeL does capability/data-flow, Progent does privilege, IsolateGPT does isolation, DataSentinel does detection, GuardAgent does policy-code generation. **Nobody has integrated all four into a single interception point that sits directly in front of the tool-call API** (i.e., a literal "gateway" in the API-gateway sense, analogous to an API security gateway/WAF). This is your core novelty opportunity.
2. **MCP-native tool poisoning defense is almost entirely unimplemented.** MCPTox and the threat-modeling paper both establish the attack surface and propose defense *sketches* (static metadata analysis, decision-path tracking, behavioral anomaly detection) but **no paper has built and evaluated a working implementation** of these specific mechanisms against real MCP servers. This is a direct, citable gap.
3. **No paper addresses tool-description integrity binding.** The 2026 attack on IsolateGPT shows that a malicious tool can rewrite the *planner's* understanding of what it's doing (Planner Manipulation) even under isolation. A cryptographic or structural mechanism to bind an agent's plan to the *original, verified* tool descriptions it was shown (detecting drift/substitution at call-time) does not yet exist.
4. **Latency/cost-aware gateway design is unstudied.** Every defense above adds either a second LLM call (DataSentinel, GuardAgent), fine-tuning requirements (StruQ/SecAlign — inapplicable to closed models), or interpreter overhead (CaMeL). **No paper systematically studies the latency/cost/security Pareto frontier** for a gateway meant to sit in the hot path of every single tool call in production. AgentSpec is the only one to report millisecond-level overhead, but for a narrower non-adversarial rule engine.
5. **Cross-session/memory-aware runtime threats are underexplored.** ASB's memory-poisoning category and PoisonedRAG both establish that persistent memory/knowledge stores are attackable, but no gateway architecture explicitly re-validates trust/provenance tags on memory *retrieved* from a prior session before it can influence a *new* tool call.
6. **No standard "provenance-tagging" protocol travels with data across the whole pipeline.** CaMeL tracks capabilities inside its own interpreter; StruQ tags at the input-formatting stage; none of these tags survive when data crosses from RAG retrieval → agent reasoning → tool call → tool output → back into agent reasoning in a general, protocol-level, MCP-compatible way. A standardized, protocol-level provenance/taint format usable across heterogeneous tools is missing.
7. **Adversarial evaluation of the guard/gateway components themselves is rare.** GuardAgent, DataSentinel, and Progent's policy-generation step are all themselves LLM-driven and therefore themselves injectable — yet almost no paper red-teams the

guard with the same rigor (GCG-style optimization attacks) it uses to red-team the *target* agent.

PART 6 — NOVEL RESEARCH IDEAS (not already solved)

1. **"Capability-Aware MCP Gateway"** — Extend CaMeL's capability/data-flow tracking to operate at the MCP protocol layer itself (intercepting `tools/list` and `tools/call` JSON-RPC messages), so the gateway is tool-framework-agnostic (works with any MCP-compliant client: Claude Desktop, IDE agents, etc.) rather than requiring a custom interpreter per agent framework, as CaMeL currently does.
2. **Tool-Description Integrity Hashing / Signing** — Require MCP servers (or a trusted registry) to cryptographically sign tool descriptions; the gateway verifies the signature and diff-checks the description shown to the planner against the description actually registered, closing the "Planner Manipulation" gap demonstrated against IsolateGPT.
3. **Two-Stage Detect-then-Enforce Pipeline with Cost-Aware Escalation** — A lightweight, cheap first-pass heuristic/classifier (fast path, catches the majority of attacks at low latency) that only escalates to an expensive DataSentinel-style adversarially-robust detector or a GuardAgent-style code-generation check when the fast path flags ambiguity — directly targeting Gap #4 above (latency/cost Pareto frontier), which no existing paper studies.
4. **Cross-Tool-Call Behavioral Anomaly Detection** — Most defenses evaluate a single tool call in isolation. A sequence-level model (e.g., a lightweight sequence classifier or rule engine, in the spirit of AgentSpec's trigger/predicate DSL) that flags anomalous *sequences* of tool calls (e.g., "read file → immediately call external HTTP tool" — a canonical exfiltration pattern) even when each individual call looks benign, directly extending AgentSpec toward Gap #7 (trajectory-based foresight, which the AgentSpec authors themselves flag as future work).
5. **Provenance-Preserving Protocol Envelope** — Define and prototype a lightweight metadata envelope (a "taint tag" carried in tool-call arguments/results) that survives the full round trip: RAG retrieval → LLM context → tool call → tool output → back into context, so that any component in the pipeline (not just your gateway) can check "was this value ultimately derived from untrusted, attacker-influenced content?" without needing a bespoke interpreter like CaMeL's. This generalizes CaMeL's capability idea into an interoperable format.
6. **Adversarially-Robust Guard-Agent (attacking the attacker-blocker)** — Apply DataSentinel's minimax/game-theoretic training methodology specifically to harden GuardAgent- and Progent-style policy-generation LLMs against prompt injection targeting *them* — an unaddressed compositional vulnerability (Gap #7).
7. **Tool-Poisoning-Aware Least-Privilege Auto-Policy Generation** — Combine Progent's SMT-verified privilege DSL with MCPTox's real-world poisoned-tool corpus

to *automatically* derive conservative default privilege policies per MCP server/tool at *registration time* (before any agent ever uses it), rather than per-task at runtime — shifting some of Progent's LLM-driven bootstrap risk to a one-time, auditable registration-time analysis.

PART 7 — PAPERS TO COMPARE AGAINST IN EVALUATION

Structure your evaluation exactly as the field does, so reviewers can immediately calibrate your numbers:

Primary baselines (system-level defenses — your direct competitors):

- CaMeL (Defeating Prompt Injections by Design) — report ASR and Utility on AgentDojo, matching their exact metric definitions
- Progent — report ASR/Utility on AgentDojo **and** ASB **and** AgentPoison, exactly as they did, for apples-to-apples three-benchmark comparison
- IsolateGPT / SecGPT — compare architecturally and, if feasible, empirically on shared attack scenarios (app compromise, data stealing)
- GuardAgent — compare on EICU-AC and Mind2Web-SC if your gateway is general enough to run those scenarios
- MELON (masked re-execution/tool comparison defense) — another AgentDojo-evaluated system-level defense worth a direct comparison row in your results table

Detection baselines (if your gateway includes a detection component):

- DataSentinel — accuracy + adaptive-attack robustness
- ProtectAI DeBERTa-v3 prompt-injection classifier — the standard "weak baseline" almost every paper includes
- PromptArmor — recent (2025) simple-but-effective LLM-as-guardrail baseline

Model-level baselines (to justify why a gateway is still necessary even if these exist):

- StruQ / SecAlign — show your gateway adds value **on top of** or **instead of** these when fine-tuning access isn't available (closed-weight models)
- Instruction Hierarchy fine-tuned models — same framing

Benchmarks to run on (your evaluation suite):

- AgentDojo (primary — utility + security)
- InjecAgent (secondary — cross-validation, different context lengths)
- Agent Security Bench / ASB (broadest attack-class coverage: DPI, IPI, memory poisoning, PoT backdoors)
- MCPTox (primary tool-poisoning-specific benchmark, real MCP servers)
- PoisonedRAG scenarios (if your gateway covers RAG-fed agents)

Adversarial stress test:

- GCG-style optimized suffixes (Zou et al.) injected into tool outputs/descriptions, to demonstrate your gateway is evaluated against adaptive, not just static, attackers — this is the single most common reviewer critique of new defense papers that skip it.

Summary Table (quick reference)

#	Paper	Venue	Year	Tier	Verdict
1	Not What You've Signed Up For	AISec'23 / ACM CCS workshop	2023	Tier-1 adjacent	MUST READ
2	The Instruction Hierarchy	OpenAI / arXiv	2024	Industry landmark	MUST READ
3	Universal & Transferable Adversarial Attacks (GCG)	arXiv	2023	Highly influential	MUST READ
4	AgentDojo	NeurIPS 2024	2024	Tier-1 AI	MUST READ
5	InjecAgent	ACL Findings 2024	2024	Tier-1 NLP	MUST READ
6	Agent Security Bench (ASB)	ICLR 2025	2025	Tier-1 AI	MUST READ
7	ToolEmu	ICLR 2024 (Spotlight)	2024	Tier-1 AI	IMPORTANT
8	PoisonedRAG	USENIX Security '25	2025	Tier-1 Security	IMPORTANT
9	CaMeL (Defeating Prompt Injections by Design)	Google DeepMind / arXiv	2025	Industry landmark	MUST READ
10	Progent	arXiv	2025	Emerging / high-impact	MUST READ
11	IsolateGPT	NDSS 2025	2025	Tier-1 Security	MUST READ
12	GuardAgent	ICML 2025	2025	Tier-1 AI	MUST READ
13	AgentSpec	ICSE 2026	2026	Tier-1 SE	IMPORTANT
14	DataSentinel	IEEE S&P 2025	2025	Tier-1 Security	MUST READ
15	Formalizing and Benchmarking Prompt Injection Attacks and Defenses	USENIX Security '24	2024	Tier-1 Security	MUST READ

#	Paper	Venue	Year	Tier	Verdict
16	StruQ	USENIX Security '25	2025	Tier-1 Security	IMPORTANT
17	SecAlign	ACM CCS 2025	2025	Tier-1 Security	IMPORTANT
18	MCP Threat Modeling / Tool Poisoning	MDPI journal	2026	Journal (Q-tier)	MUST READ (scope-fit)
19	MCPTox	arXiv	2025	Emerging benchmark	MUST READ
20	Open Challenges in Multi-Agent Security	arXiv	2025	Position paper	OPTIONAL

End of literature review. Recommend re-running MCPTox/tool-poisoning searches periodically — this is the fastest-moving sub-area (multiple new arXiv papers per month as of mid-2026), and several MUST READ arXiv-only entries above (CaMeL, Progent, MCPTox) may have since moved to Tier-1 venue proceedings.